```
(*
                              CS51 Lab 9
                          Substitution Semantics
 *)
(*
                                SOLUTION
 *)

(*======================================================================
Readings:

    This lab builds on material from Chapter 13 of the textbook
    <http://book.cs51.io>, which should be read before the lab session.

Objective:

    In this lab, you'll gain practice with understanding and generating
    substitution semantic derivations, along with the formal
    definitions of free variables and substitution.
======================================================================*)

(*======================================================================
Part 1: Substitution semantics derivation

In this part of the lab, you'll work out the formal derivation of the
substitution semantics for the expression

    let x = 3 + 5 in
    (fun x -> x * x) (x - 2)

according to the semantic rules presented in Chapter 13.

        ****************************************
        FOR THIS LAB YOU WILL NEED PENCIL AND PAPER
        OR SOME SIMILAR WRITING SYSTEM.
        ****************************************

Before beginning, what should this expression evaluate to? Test out
your prediction in the OCaml REPL. *)

(* SOLUTION: The expression evaluates to 36:

    # let x = 3 + 5 in
        (fun x -> x * x) (x - 2)  ;;
    - : int = 36
*)

(* The exercises will take you through the derivation stepwise, so
that you can use the results from earlier exercises in the later
exercises.

By way of example, we do the first couple of exercises for you to give
```

```
you the idea.

    ****************************************************************
    NOTE:

        When we ask for a derivation using a particular set of rules
        -- such as the substitution rules in Figure 13.4 or the semantic
        rules in Figure 13.5 -- the derivation should be explicit about
        that particular set of rules, but should not explicitly show the
        use of any other rules. For instance, in Exercise 3, the solution
        shows the semantic rules (Figure 13.5) in the derivation
        explicitly. Implicitly, it's making use of the substitution rules
        (Figure 13.4) by its appeal to Exercise 2.
    ****************************************************************

.....................................................................
Exercise 1. Carry out the derivation for the semantics of the
expression `3 + 5`, using the semantics rules in Figure 13.5.
.....................................................................*)

(* SOLUTION:

    3 + 5 ⇓
          | 3 ⇓ 3          (R_int)
          | 5 ⇓ 5          (R_int)
          ⇓ 8              (R_+)

   (We try to mimic the notation for semantic rules and derivations
   from the textbook, but the appearance is imperfect.)

   This derivation was actually given in the reading in Section
   13.1. We've annotated each line with the semantic rule that it
   uses. You should do that too below.
 *)

(*.....................................................................
Exercise 2. What is the result of the following substitution according
to the definition in Figure 13.4?

    (x + 5) [x ↦ 3]
.....................................................................*)

(* SOLUTION: Carrying out each step in the derivation:

    (x + 5) [x ↦ 3]
        = x [x ↦ 3] + 5 [x ↦ 3]      (by 13.10)
        = 3 + 5 [x ↦ 3]              (by 13.8)
        = 3 + 5                      (by 13.7)

   Again, we've labeled each line with the number of the equation that
   was used from the set of equations for substitution in Figure
   13.4. You should do that too.

   NOTICE that in carrying out these substitution derivations, we use
```

```
   the = operator. The idea is that the expression on the left of the
   = is *the same as* (equal to) the expression on the right. For
   instance, the expression indicated by (x + 5) [x ↦ 3] (the
   expression resulting from substituting 3 for x in x + 5 *is* the
   expression 3 + 5. The = relation differs from the ⇓ ("evaluates
   to") relation. In particular, the former is symmetric; the latter
   is not.

   NOTICE that the result of substituting `3` for `x` in `x + 5` is
   `3 + 5` and not `8`. Do you understand why? *)

(*.....................................................................
Exercise 3. Carry out the derivation for the semantics of the
expression `let x = 3 in x + 5`, using the semantics rules in Figure
13.5.
.....................................................................*)

(* SOLUTION:

    let x = 3 in x + 5 ⇓
                       | 3 ⇓ 3          (R_int)
                       | 3 + 5 ⇓ 8      (Exercises 2 and 1)
                       ⇓ 8              (R_let)

   Note the labeling of one of the steps with the prior results from
   previous exercises. *)

(* Now it's your turn. We recommend doing these exercises with
pencil on paper, rather than typing them in.

.....................................................................
Exercise 4. Carry out the derivation for the semantics of the
expression `8 - 2`, using the semantics rules in Figure 13.5.
.....................................................................*)

(* SOLUTION:

    8 - 2 ⇓
          | 8 ⇓ 8         (R_int)
          | 2 ⇓ 2         (R_int)
          ⇓ 6             (R_-)
 *)

(*.....................................................................
Exercise 5. Carry out the derivation for the semantics of the
expression `6 * 6`, using the semantics rules in Figure 13.5.
.....................................................................*)

(* SOLUTION:

    6 * 6 ⇓
          | 6 ⇓ 6          (R_int)
          | 6 ⇓ 6          (R_int)
          ⇓ 36             (R_* )
```

```
 *)

(*.....................................................................
Exercise 6. What is the result of the following substitution according
to the definition in Figure 13.4?

    (x * x) [x ↦ 6]
.....................................................................*)

(* SOLUTION:

    (x * x) [x ↦ 6]
        = x [x ↦ 6] * x [x ↦ 6]     (by 13.10)
        = 6 * 6                     (by 13.8 twice)
 *)

(*.....................................................................
Exercise 7. The set of 11 equations defining substitution in Figure
13.4 has an equation for function application. You'll need this
equation in some exercises below. Without looking at Figure 13.4,
what do you think such an equation should look like? Check your
understanding against Figure 13.4.
.....................................................................*)

(*    (Q R)[x ↦ P] = Q[x ↦ P]  R[x ↦ P]       *)

(*.....................................................................
Exercise 8. What is the result of the following substitution according
to the definition in Figure 13.4?

    ((fun x -> x * x) (x - 2)) [x ↦ 8]
.....................................................................*)

(* SOLUTION:

    ((fun x -> x * x) (x - 2)) [x ↦ 8]
        = ((fun x -> x * x) [x ↦ 8]) ((x - 2) [x ↦ 8])    (by 13.11)
        = (fun x -> x * x) ((x - 2) [x ↦ 8])              (by 13.12)
        = (fun x -> x * x) (x [x ↦ 8] - 2 [x ↦ 8])        (by 13.10)
        = (fun x -> x * x) (8 - 2 [x ↦ 8])                (by 13.8)
        = (fun x -> x * x) (8 - 2)                        (by 13.7)
 *)

(*.....................................................................
Exercise 9. Carry out the derivation for the semantics of the
expression

    (fun x -> x * x) (8 - 2)

using the semantics rules in Figure 13.5.
.....................................................................*)

(* SOLUTION:

```

```
    (fun x -> x * x) (8 - 2)
         ⇓
         | fun x -> x * x ⇓ fun x -> x * x    (R_fun)
         | 8 - 2 ⇓ 6                         (Exercise 4)
         | 6 * 6 ⇓ 36                        (Exercises 6 and 5)
         ⇓ 36                                (R_app)
 *)

(*.....................................................................
Exercise 10. Finally, carry out the derivation for the semantics of
the expression

    let x = 3 + 5 in (fun x -> x * x) (x - 2)

using the semantics rules in Figure 13.5.
.....................................................................*)

(* SOLUTION:

    let x = 3 + 5 in (fun x -> x * x) (x - 2)
         ⇓
         | 3 + 5 ⇓ 8                              (Exercise 1)
         | (fun x -> x * x) (8 - 2) ⇓ 36          (Exercise 8)
         ⇓ 36                                    (R_let)
 *)

(*=====================================================================
Part 2: Pen and paper exercises with the free variables and
substitution definitions

In this part, you'll get more practice using the definitions of FV and
substitution from the textbook (Figure 13.4). Feel free to jump ahead
to later problems if you "get it" and are finding the exercises
tedious. *)

(*.....................................................................
Exercise 11: Use the definition of FV to derive the set of free
variables in the expressions below. Show all steps using pen and
paper. (You can see an example derivation for

       FV(fun y -> f (x + y))

in Section 13.3.2 of the textbook.)

1. let x = 3 in let y = x in f x y

2. let x = x in let y = x in f x y

3. let x = y in let y = x in f x y

4. let x = fun y -> x in x
.....................................................................*)

(* SOLUTION:
```

```
Here are derivations for the free variable sets for each of the
expressions. We sometimes take multiple steps of the free variables
rules or set operations, as listed, to make the derivations a bit more
concise. Uses of the set operations union and difference are notated
as U and -, respectively.

1. let x = 3 in let y = x in f x y

      FV(let x = 3 in let y = x in f x y)
      = (FV(let y = x in f x y) - {x}) U FV(3)  (13.6)
      = (FV(let y = x in f x y) - {x})          (13.1), U
      = (FV(f x y) - {y}) U FV(x)) - {x}        (13.6)
      = ({f, x, y} - {y}) U {x}) - {x}          (13.4), (13.4), -, (13.2)
      = ({f, x} U {x}) - {x}                    -
      = {f, x} - {x}                            U
      = {f}                                     -

2. let x = x in let y = x in f x y

      FV(let x = x in let y = x in f x y)
      = (FV(let y = x in f x y) - {x}) U FV(x)  (13.6)
      = (FV(f x y) - {y}) U FV(x)) - {x} U {x}  (13.6), (13.2)
      = (({f, x, y} - {y}) U {x}) - {x}) U {x}  (13.4), (13.4), U, (13.2)
      = (({f, x} U {x}) - {x}) U {x}            -
      = ({f, x} - {x}) U {x}                    U
      = {f, x}                                  -, U

3. let x = y in let y = x in f x y

      FV(let x = y in let y = x in f x y)
      = (FV(let y = x in f x y) - {x}) U FV(y)  (13.6)
      = (FV(f x y) - {y}) U FV(x)) - {x} U {y}  (13.6), (13.2)
      = (({f, x, y} - {y}) U {x}) - {x}) U {y}  (13.4), (13.4), U, (13.2)
      = (({f, x} U {x}) - {x}) U {y}            -
      = ({f, x} - {x}) U {y}                    U
      = {f, y}                                  -, U

4. let x = fun y -> x in x

      FV(let x = fun y -> x in x)
      = (FV(x) - {x}) U FV(fun y -> x)          (13.6)
      = ({x} - {x}) U (FV(x) - {y})             (13.2), (13.5)
      = {} U {x}                                -, (13.2), -
      = {x}                                     U

 *)

(*..............................................................
Exercise 12: What expressions are specified by the following
substitutions? Show all the steps as per the definition of
substitution given in the textbook, Figure 13.4.

1. (x + 1)[x ↦ 50]
```

```
2. (x + 1)[y ↦ 50]

3. (x * x)[x ↦ 2]

4. (let x = y * y in x + x)[x ↦ 3]

5. (let x = y * y in x + x)[y ↦ 3]

.....................................................................*)

(* SOLUTION:

1. (x + 1)[x ↦ 50]
      = x[x ↦ 50] + 1[x ↦ 50]         (by 13.10)
      = 50 + 1[x ↦ 50]                (by 13.8)
      = 50 + 1                        (by 13.7)

   If you came up with the answer 51, you're confusing the
   substitution operation, which operates purely syntactically to
   replace portions of an expression, with evaluation, which
   simplifies an expression to one that is semantically equivalent but
   can be syntactically quite different.

   In case you were wondering where the parentheses around (x + 1)
   went, they disappear because those were really just in the concrete
   syntax to notate the abstract syntax, which is what the
   substitution operation operates on.

2. (x + 1)[y ↦ 50]
      = x[y ↦ 50] + 1[y ↦ 50]
      = x + 1

3. (x * x)[x ↦ 2]
      = x[x ↦ 2] * x[x ↦ 2]
      = 2 * 2

4. (let x = y * y in x + x)[x ↦ 3]
      = let x = (y * y)[x ↦ 3] in x + x            (by 13.15)
      = let x = y[x ↦ 3] * y[x ↦ 3] in x + x       (by 13.10)
      = let x = y * y in x + x                     (by 13.9 twice)

5. (let x = y * y in x + x)[y ↦ 3]
      = let x = (y * y)[y ↦ 3] in (x + x)[y ↦ 3]   (by 13.16)
      = let x = y[y ↦ 3] * y[y ↦ 3] in
        x[y ↦ 3] + x[y ↦ 3]                        (by 13.10 twice)
      = let x = 3 * 3 in x[y ↦ 3] + x[y ↦ 3]       (by 13.8 twice)
      = let x = 3 * 3 in x + x                     (by 13.9 twice)
 *)

(*.....................................................................
Exercise 13: For each of the following expressions, derive its final
value using the evaluation rules in the textbook. Show all steps using
pen and paper, and label them with the name of the evaluation rule
```

```
used. Where an expression makes use of the evaluation of an earlier
expression, you don't need to rederive the earlier expression's value;
just use it directly.

1. 2 * 25

2. let x = 2 * 25 in x + 1

3. let x = 2 in x * x

4. let x = 51 in let x = 124 in x

.....................................................................*)
(* SOLUTION:

1. 2 * 25 ⇓
          | 2 ⇓ 2              (R_int)
          | 5 ⇓ 5              (R_int)
          ⇓ 50                 (R_* )

2. let x = 2 * 25 in x + 1 ⇓
                           | 2 * 25 ⇓ 50       (from Exercise 13.1)
                           | 50 + 1 ⇓
                           |        | 50 ⇓ 50  (R_int)
                           |        | 1 ⇓ 1    (R_int)
                           |        ⇓ 51       (R_+)
                           ⇓ 51                (R_let)

(The 50 + 1 is the result of (x + 1)[x ↦ 50], as per Exercise 12.1.)

3. let x = 2 in x * x ⇓
                      | 2 ⇓ 2
                      | 2 * 2 ⇓
                      |       | 2 ⇓ 2
                      |       | 2 ⇓ 2
                      |       ⇓ 4
                      ⇓ 4

(The 2 * 2 is the result of (x * x)[x ↦ 2], as per Exercise 12.3.)

4. let x = 51 in let x = 124 in x ⇓
                                  | 51 ⇓ 51
                                  | let x = 124 in x ⇓
                                  |                  | 124 ⇓ 124
                                  |                  | 124 ⇓ 124
                                  |                  ⇓ 124
                                  ⇓ 124
 *)

(*=====================================================================
Part 3: Implementing a simple arithmetic language.

You will now implement a simple language for evaluating `let` bindings
and arithmetic expressions. Recall the following syntax for such a
```

```
language from the textbook.

<binop> ::= + | - | * | /
<var>   ::= x | y | z | ...
<expr>  ::= <integer>
          |  <var>
          |  <expr1> <binop> <expr>
          |  let <var> = <expr_def> in <expr_body>

..........................................................................
Exercise 14: We've provided below type definitions that allow for
expressions implementing this syntax. Augment the type definitions to
allow for other binary operations (at least `Minus` and `Times`) and
for unary operations (at least Negate). Hint: Don't forget to extend
the type definition of `expr` to support unary operations as well.

When you're done, you should be able to specify expressions such as
the following:

    # Int 3 ;;
    - : expr = Int 3
    # Binop (Plus, Int 3, Var "x") ;;
    - : expr = Binop (Plus, Int 3, Var "x")
    # Unop (Negate, Int 3) ;;
    - : expr = Unop (Negate, Int 3)
    # Let ("x", Int 3, Binop (Plus, Int 3, Var "x")) ;;
    - : expr = Let ("x", Int 3, Binop (Plus, Int 3, Var "x"))
..........................................................................*)

type varspec = string ;;

type binop =
  | Plus
  | Minus
  | Times
  | Divide ;;

type unop =
  | Negate ;;

type expr =
  | Int of int
  | Var of varspec
  | Binop of binop * expr * expr
  | Unop of unop * expr
  | Let of varspec * expr * expr ;;

(*..........................................................................
Exercise 15: Write a function `free_vars : expr -> varspec Set.t` that
returns a set of `varspec`s corresponding to the free variables in the
expression.

The free variable rules in this simple language are a subset of those
found in Figure 13.4, but we encourage you to first try to determine
```

```
the rules on your own, consulting the textbook only as necessary.

To handle all of the set processing in the free variable rules --
unions and differences and so on -- we've provided a `VarSet` module
built using OCaml's `Set.Make` functor. (More documentation on the
`Set.Make` functor can be found at
<https://v2.ocaml.org/api/Set.Make.html>.)

You should get behavior such as this, in calculating the free
variables in the expression

    let x = x + y in z * 3      :

    # VarSet.elements
        (free_vars (Let ("x",
                         Binop (Plus, Var "x", Var "y"),
                         Binop (Times, Var "z", Int 3)))) ;;
    - : Lab9.VarSet.elt list = ["x"; "y"; "z"]
.....................................................................*)

module VarSet = Set.Make (struct
                            type t = varspec
                            let compare = String.compare
                          end) ;;

let rec free_vars (exp : expr) : VarSet.t =
  match exp with
  | Var x -> VarSet.singleton x
  | Int _ -> VarSet.empty
  | Unop(_, arg) -> free_vars arg
  | Binop(_, arg1, arg2) ->
     VarSet.union (free_vars arg1) (free_vars arg2)
  | Let(x, def, body) ->
     VarSet.union (free_vars def) (VarSet.remove x (free_vars body))
;;

(*.....................................................................
Exercise 16: Write a function `subst : expr -> varspec -> expr ->
expr` that performs substitution, that is, `subst p x q` returns the
expression that is the result of substituting `q` for the variable `x`
in the expression `p`.

The necessary substitution rules for this simple language are as
follows:

m[x ↦ P] = m                          (where m is some integer value)

x[x ↦ P] = P

y[x ↦ P] = y                    (where x and y are distinct variables)

(~- Q)[x ↦ P] = ~- Q[x ↦ P]        (and similarly for other unary ops)

(Q + R)[x ↦ P] = Q[x ↦ P] + R[x ↦ P]
```

```
                                    (and similarly for other binary ops)

(let x = Q in R)[x ↦ P] = let x = Q[x ↦ P] in R

(let y = Q in R)[x ↦ P] = let y = Q[x ↦ P] in R[x ↦ P]
                                  (where x and y are distinct variables)

You should get the following behavior:

    # let example = Let ("x", Binop (Plus, Var "x", Var "y"),
                              Binop (Times, Var "z", Var "x")) ;;
    val example : Lab9.expr =
      Let ("x", Binop (Plus, Var "x", Var "y"), Binop (Times, Var "z", Var "x"))
    # subst example "x" (Int 42) ;;
    - : Lab9.expr =
    Let ("x", Binop (Plus, Int 42, Var "y"), Binop (Times, Var "z", Var "x"))
    # subst example "y" (Int 42) ;;
    - : Lab9.expr =
    Let ("x", Binop (Plus, Var "x", Int 42), Binop (Times, Var "z", Var "x"))
.....................................................................*)

let subst (exp : expr) (var_name : varspec) (repl : expr) : expr =

  (* perform this particular substitution of variable and replacement *)
  let rec sub_this (exp : expr) : expr =
    match exp with
    | Var x -> if x = var_name then repl else exp
    | Int _ -> exp
    | Unop(op, arg) -> Unop(op, sub_this arg)
    | Binop(op, arg1, arg2) -> Binop(op, sub_this arg1, sub_this arg2)
    | Let(x, def, body) ->
       if x = var_name then Let(x, sub_this def, body)
       else Let(x, sub_this def, sub_this body) in

  sub_this exp ;;

(*.....................................................................
Exercise 17: Complete the `eval` function below. Try to implement
these functions from scratch. If you get stuck, however, a good
(though incomplete) start can be found in section 13.4.2 of the
textbook.
.....................................................................*)

(* Please use the provided exceptions as appropriate. *)
exception UnboundVariable of string ;;
exception IllFormed of string ;;

let binopeval (op : binop) (v1 : expr) (v2 : expr) : expr =
  match op, v1, v2 with
  | Plus, Int x1, Int x2 -> Int (x1 + x2)
  | Plus, _, _ -> raise (IllFormed "can't add non-integers")
  | Minus, Int x1, Int x2 -> Int (x1 - x2)
  | Minus, _, _ -> raise (IllFormed "can't subtract non-integers")
  | Times, Int x1, Int x2 -> Int (x1 * x2)
```

```
  | Times, _, _ -> raise (IllFormed "can't multiply non-integers")
  | Divide, Int x1, Int x2 -> Int (x1 / x2)
  | Divide, _, _ -> raise (IllFormed "can't divide non-integers") ;;

let unopeval (op : unop) (e : expr) : expr =
  match op, e with
  | Negate, Int x -> Int (~- x)
  | Negate, _ -> raise (IllFormed "can't negate non-integers")

let rec eval (exp : expr) : expr =
  match exp with
  | Int _ -> exp
  | Var x -> raise (UnboundVariable x)
  | Unop (op, exp1) -> unopeval op (eval exp1)
  | Binop (op, exp1, exp2) -> binopeval op (eval exp1) (eval exp2)
  | Let (x, def, body) -> eval (subst body x (eval def)) ;;

(*.....................................................................
Go ahead and test `eval` by evaluating some arithmetic expressions and
let bindings.

For instance, try the following expression, which is essentially

       let x = 6 in let y = 3 in x * y     .

# eval (Let ("x", Int 6,
                  Let ("y", Int 3,
                            Binop (Times, Var "x", Var "y")))) ;;
- : expr = Int 18

You now have a good start on the final project!
.....................................................................*)
```